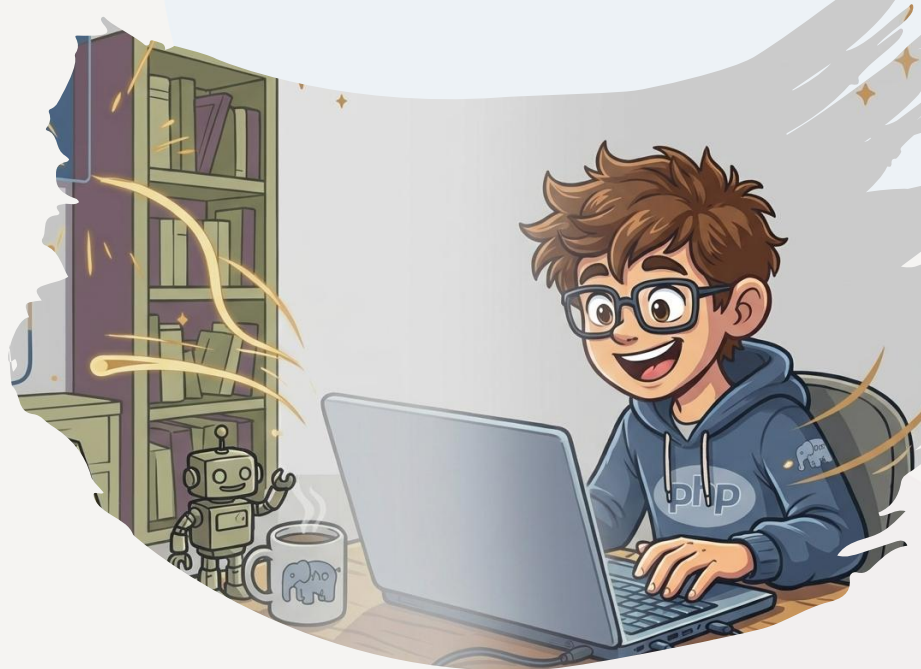


Programación VII

Programación PHP

Conceptos Fundamentales

Profesor Ramón González Tribaldos · UTPCH · I Semestre 2026



01

Sintaxis Básica en PHP

Reglas de oro para la escritura de programas en PHP.

02

Tipos de Datos en PHP

Tipos de datos y ámbito de variables, conversión de tipos de datos

03

Sentencias de E|S

Instrucciones para la interacción desde el navegador.

04

Estructuras de Control

Operadores básicos; estructuras condicionales y cíclicas de PHP.

05

Manejo de Arreglos

Tipos de arreglos; arreglos escalares, asociativos y multidimensionales; recorrido de arreglos.

06

Manejo de Funciones

Funciones para constantes, variables, cadenas, matemáticas, de conversión, fecha y hora.

Sintaxis Básica de PHP

- PHP se abre con `<?php` y se cierra con `?>`
- Cada instrucción termina con `;`
- Comentarios: `//` (una línea) o `/* */` (bloque o multilínea).
- Variables siempre empiezan con `$` (ej. `$nombre`)
- Los bloques de código usan llaves `{ }`
- **Case Sensitive:** Las variables son sensibles a mayúsculas .



Reglas de Oro

```
<?php
// La etiqueta de apertura es obligatoria
echo "¡Bienvenidos a la Misión PHP!";
/* El comando echo muestra texto en el navegador */
?>
```



Sintaxis Básica de PHP

Los identificadores son los nombres que se utilizan para identificar variables, funciones, clases, etc.

Los identificadores siempre empiezan con \$ (ej. \$nombre).

```
$body_temp = 98.6 ;
```

Pueden empezar por una letra o el carácter *underscore* (guion bajo _).

Case Sensitive: Distingue entre letras mayúsculas y minúsculas (\$total ≠ \$Total).

No pueden usarse palabras reservadas. (ej. If, while, echo).

Se utiliza la palabra reservada **const** para definir valores que no cambian durante la ejecución del script .

Como buena práctica, una constante se suele escribir en mayúscula. (ej. const PI =3.1416).



Identificadores

Sintaxis Básica de PHP



Identificadores

```
<?php
```

```
// Asignación básica
```

```
$nombre = "María";
```

```
$edad = 25;
```

```
$activo = true;
```

```
// Ámbito global
```

```
$contador = 0;
```

Prefijo \$

Toda variable empieza con \$. El nombre puede contener letras, dígitos y _ pero NO puede empezar con dígito.

Tipado dinámico

No se declara el tipo. PHP lo asigna automáticamente según el valor asignado y puede cambiar en tiempo de ejecución.

Estructura de un programa en PHP

¿Cómo se organiza un archivo PHP?

Etiquetas de apertura y cierre

`<?php ... ?>` delimitan el código. La etiqueta de cierre es opcional en archivos puramente PHP (buena práctica omitirla).

Mezcla con HTML

PHP puede insertarse dentro de HTML. El servidor ejecuta el bloque PHP y devuelve HTML puro al navegador.

Sensibilidad a mayúsculas

Las funciones NO son sensibles (`echo` = `ECHO`), pero las variables SÍ (`$nombre` ≠ `$Nombre`).

Punto y coma obligatorio

Cada instrucción termina con `;` (su omisión genera un error de parseo).

Ejemplo completo — Reglas de escritura

```
<?php
// Comentario de una línea
/* Comentario
   multilínea */
# También es comentario

// Las funciones NO son case-sensitive
echo "Hola PHP\n";
ECHO "También funciona\n";

// Las variables SÍ son case-sensitive
$nombre = "Ana";
$Nombre = "Variable diferente";

echo $nombre; // imprime: Ana
```

Tipos de datos en PHP



Escalares

`int`

Números enteros sin decimal.
-2147483648 a 2147483647

`float`

Números con punto decimal.

`string`

Cadena de caracteres; usa `' '` o `" "`.

`bool`

true o false.



Compuestos

`array`

Colección de valores indexados.

`object`

Instancia de una clase (POO).



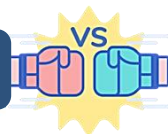
Especiales

`null`

Variable sin valor asignado.

Ámbito de variables en PHP

ÁMBITO GLOBAL



✓ **Definición:** Variables visibles desde cualquier punto del código.

✦ Se accede mediante la palabra reservada `global`.

✓ **Características:**

- Disponibles en todo el script
- Definidas fuera de funciones
- Pueden modificarse desde cualquier lugar

✦ EJEMPLO

```
$usuario = "Ana"; // variable GLOBAL

function saludar() {
    global $usuario;
    echo "Hola " . $usuario;
}
```

ÁMBITO LOCAL

✓ **Definición:** Variables solo visibles dentro de una función.

✓ **Características:**

- No se pueden acceder desde fuera
- Se crean y destruyen con la función
- Evitan conflictos de nombres

✦ EJEMPLO

```
function calcular() {
    $resultado = 10 + 5; // variable LOCAL
    return $resultado;
}
// echo $resultado; ✗ ERROR - No visible fuera
```


Variables Superglobales en PHP

- Las **variables superglobales** están predefinidas por PHP y son accesibles en cualquier contexto sin usar la palabra global.
- No es necesario definirlas antes de usarlas.



Variable	Descripción
\$_GLOBALS	Contiene variables disponibles en el ámbito global.
\$_SERVER	Contiene información del servidor como cabeceras, nombre del host, etc.
\$_GET	Contiene datos enviados por el método GET o por URL. 
\$_POST	Contiene datos enviados por el método POST. 
\$_FILES	Contiene datos de archivos que se están subiendo al servidor.
\$_COOKIE	Contiene Cookies HTTP. 
\$_SESSION	Contiene variables de sesión. 
\$_REQUEST	Contiene datos de una petición HTTP.
\$_ENV	Contiene variables del entorno del servidor web.

Conversión de tipos de datos

- PHP es un lenguaje de tipado débil y dinámico. Esto significa que las variables no tienen un tipo fijo y pueden cambiar según el contexto.
- La conversión de tipos puede ser explícita (type casting) o implícita (automática).



Conversión
Explícita

Tipo de función	Código de ejemplo	Resultado	Explicación
A entero	(int) "\$12.99"	12 (int)	Trunca decimales, toma parte numérica inicial.
A float	(float) "3.1416"	3.1416 (float)	Convierte cadena decimal a número real.
A string	(string) 100	"100" (string)	Número a representación textual.
A booleano (false)	(bool) 0	false	0, 0.0, "", "0", [], null → false.
A booleano (true)	(bool) 42	true	Cualquier otro valor no vacío/cero → true.

Conversión de tipos de datos

- La conversión implícita ocurre cuando PHP detecta que un operador requiere un tipo de dato distinto al original.



**Conversión
Implícita**

Operación	Ejemplo	Resultado (tipo)	Explicación breve
Suma string numérico + número	"10" + 5	15 (int)	PHP convierte "10" a entero.
Suma string decimal + float	"10.5" + 1.5	12.0 (float)	String a float automático.
Suma string no numérico + número	"Hola" + 3	3 (int)	"Hola" → 0 antes de sumar.
Booleano en suma	true + 1	2 (int)	true → 1, false → 0.
Concatenación con número	"Total: " . 42	"Total: 42" (string)	Número convertido a string.
Comparación flexible	if ("5" == 5)	true (bool)	Compara solo valor, no tipo.

Funciones de entrada de datos

- PHP puede recibir datos desde consola (CLI).



Entrada por consola

fgets(STDIN) — Lee una línea desde la entrada estándar.

```
echo "¿Cómo te llamas? ";  
$nombre = trim(fgets(STDIN));  
echo "Hola, $nombre";
```

⚙ Usar trim() para eliminar el salto de línea final.

Funciones de entrada de datos

- PHP puede recibir datos desde formularios web.



Entrada desde formularios web

`$_GET['nombre']` — Datos enviados por método GET (visible en URL).

```
// URL: https://ejemplo.com/pagina.php?usuario=Ana&edad=25
$user = $_GET['usuario']; // "Ana"
$edad = $_GET['edad']; // "25"
```

`$_POST['nombre']` — Datos enviados por método POST (cuerpo HTTP, no visible en URL).

```
// Desde formulario HTML con method="POST"
$email = $_POST['correo'];
```

Funciones de salida de datos

- Muestran información en pantalla (navegador o consola).



◆ echo

echo — Construcción del lenguaje. Rápida, sin valor de retorno.

```
echo "Hola mundo";  
echo "Línea 1  
", "Línea 2"; // Múltiples parámetros
```

◆ print

print — Similar a echo, pero siempre retorna 1.

```
print "Hola";  
$resultado = print "Texto"; // $resultado = 1
```

Funciones de salida de datos

- Muestran información en pantalla (navegador o consola).



◆ printf

printf – Salida formateada (similar a C).

```
printf("Entero: %d, Flotante: %.2f", 42, 3.1415);
```

◆ var_dump()

var_dump() – Muestra tipo y valor de una variable (debug).

```
$texto = "PHP es genial";  
var_dump($texto);  
$numero = 42;  
var_dump($numero);
```

Operadores Aritméticos

Operador	Operación
+	Adición
-	Sustracción
*	Multiplicación
/	División
%	Módulo
**	Exponenciación



Operadores Relacionales

Operador	Prueba comparativa
==	Igualdad
===	Identidad
!==	No identidad
!= < >	Desigualdad
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que



Operadores Lógicos

Operador		Operación
and	&&	AND Lógico
or		OR Lógico
!		NOT Lógico



Prioridad de Operadores

Operador	Descripción
**	Exponenciación
*	Producto
/	División
%	Módulo
+	Suma
-	Resta

"Los paréntesis tienen la máxima prioridad en la evaluación de expresiones, permitiendo definir explícitamente el orden de ejecución, mientras que el resto de operadores siguen su jerarquía preestablecida."



**Prioridad de
operadores**

Sentencias Condicionales

Condicional IF:

- Ejecuta un bloque de código si la condición es verdadera.



```
if ( expresión-condicional )  
{  
    instrucción-a-ejecutar-cuando-sea-verdadera ;  
    instrucción-a-ejecutar-cuando-sea-verdadera ;  
    instrucción-a-ejecutar-cuando-sea-verdadera ;  
}
```

```
<?php  
if ( 4 > 2 ) { echo '<p>Yes, 4 is greater than 2 <br>' ; }
```



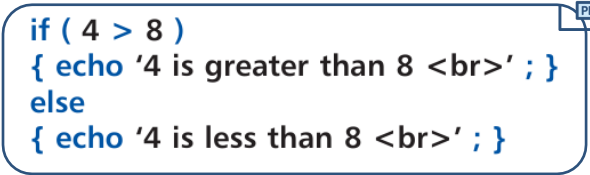
Sentencias Condicionales

Condicional IF-ELSE:

- Ejecuta un bloque de código si la condición es verdadera ; de lo contrario, ejecuta otro bloque.



```
if ( (expresión-condicional) )
{
    instrucción-a-ejecutar-cuando-sea-verdadera ;
    instrucción-a-ejecutar-cuando-sea-verdadera ;
    instrucción-a-ejecutar-cuando-sea-verdadera ;
}
else
{
    instrucción-a-ejecutar-cuando-sea-falsa ;
    instrucción-a-ejecutar-cuando-sea-falsa ;
    instrucción-a-ejecutar-cuando-sea-falsa ;
}
```



```
if ( 4 > 8 )
{ echo '4 is greater than 8 <br>' ; }
else
{ echo '4 is less than 8 <br>' ; }
```

Sentencias Condicionales

Condicional IF-ELSEIF-ELSE:

- Permite verificar múltiples condiciones y ejecutar diferentes bloques de código según el caso.



```
if ( 4 > 8 )  
{ echo 'This test is True </p>' ; }  
elseif ( 8 > 4 )  
{ echo 'Alternative test is True </p>' ; }  
else  
{ echo 'Both tests are False </p>' ; }
```





Ejercicios Prácticos

Enunciado 1:



- Escribe un programa en PHP, que dadas las edades de 3 personas, calcule la edad promedio. Muestra el resultado con un mensaje descriptivo.

Enunciado 2:



- Escribe un programa en PHP, que dadas la base y la altura de un triángulo, muestre por pantalla el área de un triángulo. Calcule su área utilizando la fórmula:

$$\text{Área} = \frac{\text{Base} \times \text{Altura}}{2}$$

Enunciado 3:



- Escribe un programa en PHP que, dados dos números, determine cuál es el mayor de los dos.

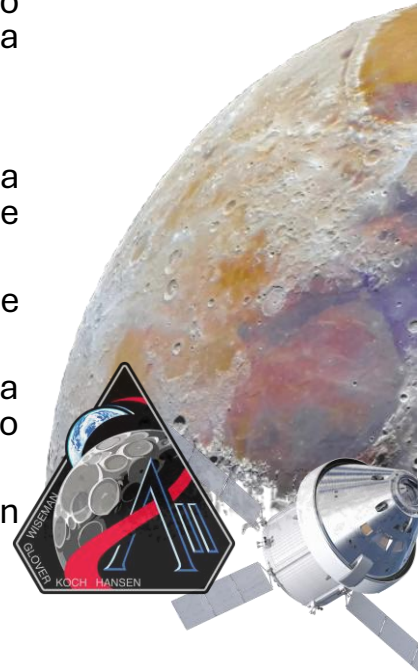


Ejercicios Prácticos

Enunciado 4: El 10 de abril de 2026, la cápsula Orión de la NASA completó su regreso a la Tierra tras el histórico sobrevuelo lunar de la misión Artemis II. Durante la reentrada a la atmósfera, la nave atraviesa distintas fases críticas según su altitud:

- Primero se separa el módulo de servicio por encima de los **400,000** pies.
- Luego, entre los **100,000** y **400,000** pies, el calor genera una capa de plasma alrededor de la cápsula que interrumpe todas las comunicaciones durante aproximadamente 6 minutos (interrupción de comunicaciones).
- Al recuperar señal por debajo de los **100,000** pies, se despliegan los paracaídas de freno alrededor de los **22,000** pies.
- Cerca de los **6,000** pies se abren los tres paracaídas principales para reducir la velocidad; y finalmente, por debajo de esa altitud, la cápsula ameriza en el océano Pacífico frente a las costas de San Diego.

Escriba un programa en PHP que, dada una variable llamada altitud, determine en cuál de las cinco fases se encuentra la cápsula e imprima el mensaje por pantalla.



Sentencias Condicionales

Condicional SWITCH-CASE:

- Permite simplificar comparaciones donde se utilizan muchos **ELSEIF**. La sentencia **switch** recibe una expresión y compara su valor con patrones dados en uno o más **case**.

```
switch ( test-value )  
{  
    case match-value : statements-to-execute-when-matched ; break ;  
    case match-value : statements-to-execute-when-matched ; break ;  
    case match-value : statements-to-execute-when-matched ; break ;  
    default : statements-to-execute-when-no-match-found ;  
}
```



Sentencias Condicionales

Conditional SWITCH-CASE:



```
<?php
$number = 2 ; $letter = 'B' ;
switch ( $letter )
{
    case 'A' : echo 'Letter is A<br>' ; break ;
    case 'B' : echo 'Letter is B<br>' ; break ;
    case 'C' : echo 'Letter is C<br>' ; break ;
    default : echo 'Letter is Unrecognized<br>' ;
}
switch ( $number )
{
    case 0 : case 1 : case 2 : echo 'Less than 3<br>' ; break ;
    default : echo '3 or more, or less than zero' ;
}
?>
```

Sentencias Condicionales

Condicional **MATCH-CASE**:

- Es una estructura condicional que permite comparar un valor contra varios patrones posibles, de una manera más limpia, precisa y segura (si se compara con la sentencia **switch**).
- Se introduce en PHP 8.0 y aplica comparación estricta.

```
$result = match ( test-value )  
{  
    match-value => return-value ,  
    match-value => return-value ,  
    match-value => return-value ,  
    default  => return-value ,  
};
```



Sentencias Condicionales

Condicional MATCH-CASE:

```
<?php
$number = 2 ; $letter = 'B' ;
echo match ( $number )
{
    1 => 'Number is One<br>' ,
    2 => 'Number is Two<br>' ,
    3 => 'Number is Three<br>' ,
    default => 'Number is Unrecognized<br>' ,
};
```





Ejercicios Prácticos

Enunciado 5:

Diseñe un script en PHP que permite identificar los códigos de estado de una petición HTTP solicitada por un usuario. Una vez leído el código, deberá mostrarse al usuario, por pantalla, la interpretación del código. Considere los siguientes códigos de uso frecuente: 200 (Petición exitosa), 403 (Petición denegada), 404 (No encontrado), 410 (No disponible), 500 (Problema inesperado).



Sentencias Cíclicas

Ciclo FOR:

- Permite ejecutar un bloque de código un número determinado de veces.



```
for ( initializer ; test-expression ; updater )  
{  
    statement-to-execute-when-true ;  
    statement-to-execute-when-true ;  
    statement-to-execute-when-true ;  
}
```



```
<?php  
for ( $i = 1 ; $i < 4 ; $i++ )  
{  
    echo "<dt>Outer loop iteration $i" ;  
    for ( $j = 1 ; $j < 4 ; $j++ )  
    {  
        echo "<dd>Inner loop iteration $j" ;  
    }  
}  
?>
```

Sentencias Cíclicas

Ciclo WHILE:

- Permite ejecutar un bloque de código repetidamente mientras una condición dada se cumpla. Cuando la condición sea Falsa, el ciclo termina.



```
initializer
while ( test-expression )
{
    statement/s-to-be-executed ;
    updater ;
}
```



```
<?php
echo '<dt>While Loop Summation : ' ;
$i = 0 ;
$total = 0 ;
while ( $i < 5 )
{ $total += ( $i + 1 ) ;
  echo "<dd>Current Iteration $i , Total is $total " ;
  $i++ ;
}
echo "<dd>Final Total sum is $total " ;
```



Ejercicios Prácticos



Enunciado 6: Mario entrena saltos sobre banderas. Su entrenamiento comienza con un número entero positivo **N** que representa la altura de la bandera en bloques.

1. En cada turno, si la altura actual es divisible por 3, Mario da un **"salto largo"** y la altura se divide entre 3.
2. Si no es divisible por 3, da un **"salto corto"** y la altura disminuye en 1.
3. El entrenamiento termina cuando la altura llega a exactamente 1 (desde allí puede tocarla con un pequeño brinco).

Escriba un script en PHP que reciba **N**, desde el navegador, muestre la secuencia completa de alturas hasta llegar a 1, y devuelva la cantidad total de pasos realizados.





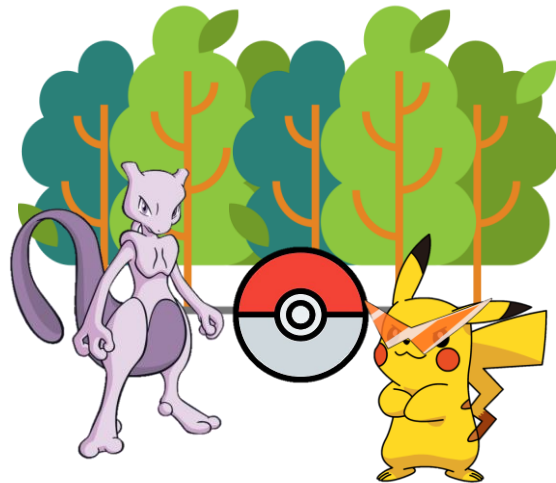
Ejercicios Prácticos



Enunciado 7: Diseña un programa en PHP que **emule** el entrenamiento de Pikachu utilizando un ciclo **MIENTRAS**. El objetivo es que Pikachu alcance el nivel 5. Pikachu comienza en nivel 1 y debe alcanzar el nivel 5 mediante batallas.

- En cada iteración, muestra el nivel actual y pregunta al usuario si Pikachu quiere pelear o huir (el usuario escribe "pelear" o "huir").
- Si pelea, Pikachu gana siempre y sube 1 nivel. Si huye, pierde 1 nivel, pero el nivel nunca puede bajar de 0 (si está en 0 y huye, se queda en 0).
- El ciclo se repite mientras el nivel sea menor a **5**. Al terminar, el programa debe mostrar el mensaje:

“¡Pikachu, estás listo para la batalla!” “¡Evolucionaste a nivel 5!”



Sentencias Cíclicas

Ciclo DO WHILE:

- Permite ejecutar un bloque de código al menos una vez antes de evaluar la condición (la comprobación ocurre al final de cada iteración).



```
initializer
do {
    statement/s-to-be-executed ;
    updater ;
}
while ( test-expression ) ;
```



```
echo '<dt>Replacement do...while Loop : ' ;
$k = 0 ;
$result = 0 ;
do
{ $result += ( $k + 1 ) ;
  echo "<dd>Attempt $k, Result is $result " ;
  $k++ ;
} while ( $k < 5 ) ;
```

Sentencias Cíclicas

Break:

- Puede utilizarse para terminar inmediatamente un bucle cuando se cumple una condición específica.
- La sentencia **break** se sitúa dentro del bloque de sentencias del bucle y va precedida de una expresión de prueba.
- Sólo afecta al bucle donde se encuentra, no a los de niveles superiores.
- Puede usar un argumento numérico para indicar cuánto niveles anidados debe interrumpir.



```
if ($i == 2 && $j == 2) {  
    break 2;  
    $k++;  
while ( $k < 5 );
```



Sentencias Cíclicas

Continue:

- Puede utilizarse para omitir una sola iteración de un bucle cuando se cumple una condición específica.
- Salta a la siguiente iteración sin terminar el bloque actual.
- La sentencia **continue** se sitúa dentro del bloque de sentencias del bucle y va precedida de una expresión de prueba.
- También puede utilizar argumento numérico.



```
<?php
for ( $i = 1; $i <= 3; $i++) {
    echo "i = $i\n";
    for ( $j = 1; $j <= 3; $j++) {
        if ( $i == 2 && $j == 2 ) {
            continue 2;
        }
        echo " j = $j\n";
    }
}
?>
```

Arreglos en PHP

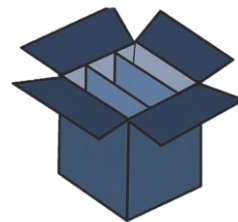
- Un arreglo en PHP es una estructura de datos que permite almacenar múltiples valores bajo un mismo nombre de variable.
- Cada valor se identifica mediante un índice o clave (key) y tiene un valor asociado.
- PHP maneja los arreglos de forma muy flexible: pueden crecer dinámicamente, mezclar tipos de datos y usar claves numéricas o de texto.



	Colombia	Perú	Ecuador	USA	Brasil
Indices	0	1	2	3	4

Tipos de Arreglos en PHP

- Se puede distinguir 3 tipos de arreglos en PHP.
 - Arreglos escalares o numéricos.
 - Arreglos asociativos.
 - Arreglos multidimensional.



Arreglos Escalares (numéricos)

- Son arreglos cuyas claves son números enteros que comienzan en 0 por defecto y se incrementan automáticamente. También se denominan arreglos indexados o numéricos.
- Ej. \$frutas = array("manzana", "pera", "uva");



Sintaxis de Arreglo Numérico

El uso de la función `array()` es el método principal para definir un arreglo de valores.

```
<?php  
$misNumeros = array(10, 20, 30, 40);  
?>
```

Función de Constructor de Arreglo

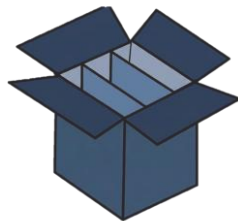
	0	1	2	3	
Valores	10	20	30	40	Tamaño del Arreglo: 4
	Índices (Implícitos)				

En un Arreglo Numérico, los índices se generan de forma implícita comenzando desde 0.

Arreglos Escalares (numéricos)

Implementaciones:

- Ejemplos de implementaciones de arreglos escalares.



```
// PHP asigna índices 0, 1, 2 automáticamente
$colores = ["rojo", "verde", "azul"];

echo $colores[0]; // rojo
echo $colores[2]; // azul

// Índice personalizado
$meses = [];
$meses[1] = "Enero";
$meses[2] = "Febrero";
$meses[3] = "Marzo";

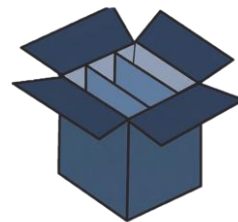
// Agregar elementos al final
$colores[] = "amarillo"; // índice 3

// Tamaño del arreglo
echo count($colores); // 4
```



Arreglos Asociativos (diccionario)

- Usan cadenas de texto como claves en lugar de números. Son equivalentes a los diccionarios o mapas en otros lenguajes.
- Cada clave es única dentro del arreglo.
- Ej. \$estudiante = ["nombre" => "Ana López", "edad" => 20];



Sintaxis de Arreglo Asociativo

El uso de la función `array()` es el método principal para definir un arreglo de valores.

```
<?php
$misEdades = array(
    "Ana" => 25,
    "Bob" => 32,
    =>
    "Charlie" => 28
);
?>
```

Constructor con asignación Clave=>Valor

Clave: 'Ana'	Clave: 'Bob'	Clave: 'Charlie'
Valor: 25	Valor: 32	Valor: 28

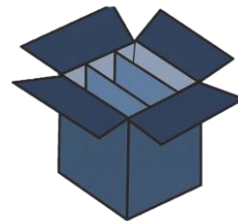
Pares Clave-Valor Cadenas Explícitas como Índices

En un Arreglo Asociativo, las claves se definen explícitamente como cadenas de texto.

Arreglos Asociativos (diccionario)

Implementaciones:

- Ejemplos de implementaciones de arreglos asociativos.



```
// Arreglo asociativo de un estudiante
$estudiante = [
    "nombre" => "Ana López",
    "edad" => 20,
    "carrera" => "Informática",
    "activo" => true
];

// Acceso por clave
echo $estudiante["nombre"]; // Ana López
echo $estudiante["carrera"]; // Informática

// Modificar un valor
$estudiante["edad"] = 21;

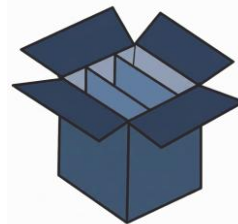
// Agregar nueva clave
$estudiante["promedio"] = 8.5;
```



Sentencia foreach

foreach:

- Es la estructura de repetición diseñada específicamente para recorrer arreglos en PHP.
- En cada iteración extrae automáticamente la clave y el valor del elemento actual, sin necesidad de manejar un índice manualmente.



```
// Sintaxis básica (solo valor)
foreach ($arreglo as $valor) {
    // código
}

// Sintaxis con clave y valor
foreach ($arreglo as $clave => $valor) {
    // código
}
```



Ejercicios Prácticos

Enunciado 8: Un docente registró las calificaciones obtenidas por cinco estudiantes en el último examen parcial: 72, 85, 90, 68 y 78 puntos. Escribe un programa en PHP que almacene esas cinco calificaciones en un arreglo escalar. Luego, recorre el arreglo sumando cada elemento para obtener el total, divide ese total entre la cantidad de elementos para calcular el promedio, e identifica la nota más alta y la más baja. Finalmente, muestra en pantalla el promedio, la calificación máxima y la calificación mínima.

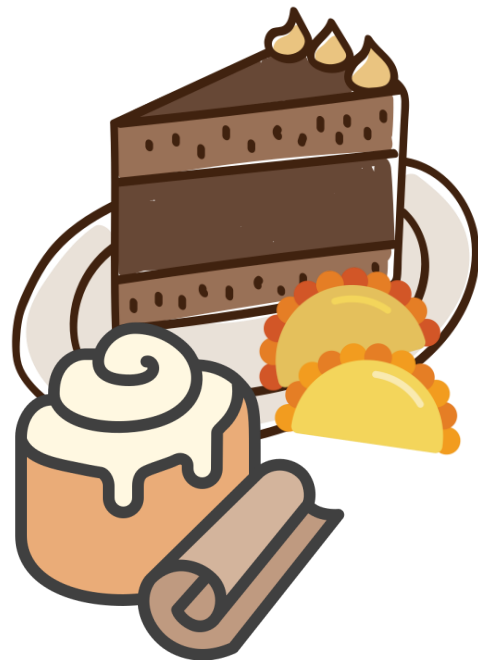
PHP tiene las funciones `max($notas)` y `min($notas)` que devuelven el mayor y menor valor de un arreglo directamente.





Ejercicios Prácticos

Enunciado 9: La pastelería Dulce Chiriquí, ubicada en el centro de David, Chiriquí, ofrece una variedad de postres y panes artesanales: Cinnamon Rolls a B/. 1.25, Mamallena a B/. 0.75, Torta de Chocolate a B/. 3.50 y Empanada de Piña a B/. 0.50. El dueño del negocio desea mostrar en la pantalla de caja el listado completo de productos con sus precios, por lo que te solicita desarrollar un script en PHP que almacene los productos en un array asociativo — usando el nombre del producto como clave y su precio como valor — y que luego utilice un ciclo foreach para recorrer el array e imprimir en pantalla, por cada producto, el mensaje: "Lleva un(a) [nombre del producto] por solo B/. [precio]", seguido de un salto de línea.

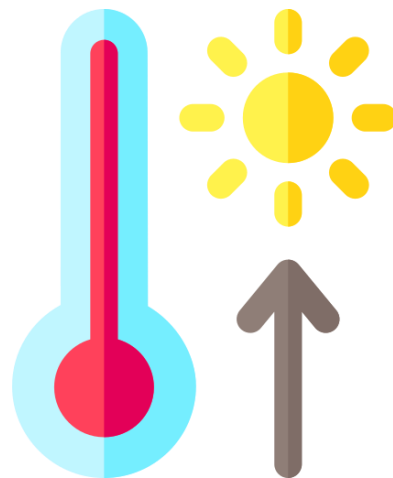




Ejercicios Prácticos

Enunciado 10: La estación meteorológica de David, Chiriquí, registró las temperaturas máximas en grados Celsius durante los últimos 7 días: 28, 31, 27, 33, 30, 29 y 32. Escribe un script en PHP que almacene esas temperaturas en un arreglo escalar y utilice un ciclo para recorrerlo. Por cada día, imprime un mensaje con el formato: *"Día [número]: [temperatura]°C"* e indica si esa temperatura está por encima o por debajo del promedio semanal. Al finalizar el recorrido, el script debe mostrar: la temperatura más alta y la más baja de la semana, el promedio semanal redondeado a dos decimales, y el porcentaje de días que superaron los 30°C.

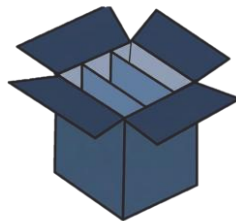
PHP tiene la función `round($variable, 2)` que permite redondear decimales.



Arreglos Multidimensionales

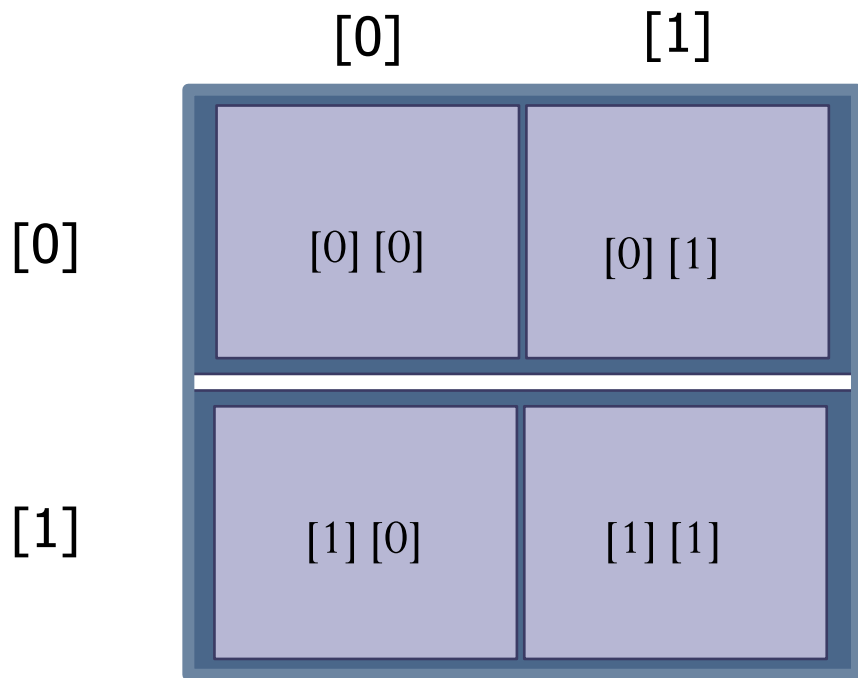
Dos Dimensiones:

- Un arreglo de dos dimensiones es un arreglo de arreglos. Cada elemento del arreglo principal contiene otro arreglo, formando una estructura de tabla (filas y columnas).
- A esta estructura también se le conoce como matrices.



```
$frutas = [  
    ["Mangos", "1lb", "$1.50"],  
    ["Fresas", "1lb", "$3.50"],  
    ["Naranjas", "1lb", "$0.80"],  
    ["Cerezas", "1lb", "$5.00"]  
];
```

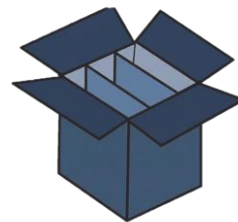
Arreglos Multidimensionales



`$matriz[0] = [valor1, valor2];`

`$matriz[1] = [valor1, valor2];`

`$matriz = [];`





Ejercicios Prácticos

Enunciado 11: Escribe un script en PHP que realice las siguientes acciones: Inicializar un array de 10 elementos con valores aleatorios entre 1 y 30. Una vez que ha inicializado el array, imprimir todos los valores que almacena.

PHP tiene la siguiente función built-in: `rand($min, $max)`: Genera un número aleatorio.



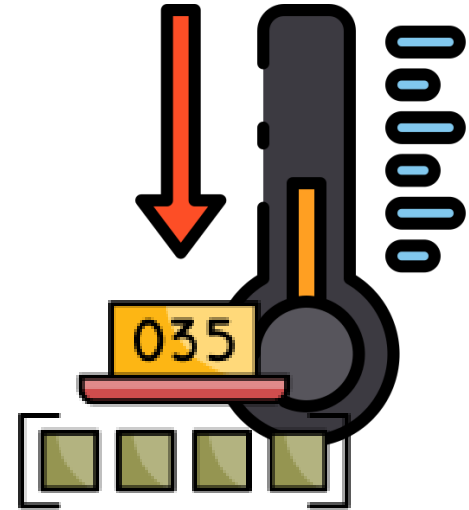


Ejercicios Prácticos

Enunciado 12: Escribe un script en PHP que gestione un arreglo de temperaturas generado automáticamente con 10 elementos, utilizando valores aleatorios comprendidos entre 1 y 30. El programa debe realizar el cálculo de la temperatura media del conjunto, así como identificar los valores máximo y mínimo presentes en el arreglo. Finalmente, el script debe mostrar en pantalla todos los resultados obtenidos.

PHP tiene las siguientes funciones built-in:

- **rand(\$min, \$max):** Genera un número aleatorio.
- **count(\$array):** Cuenta todos los elementos de un arreglo.
- **array_sum(\$array):** Suma todos los valores numéricos de un arreglo.
- **max(\$array):** Valor máximo del arreglo.
- **min(\$array):** Valor mínimo del arreglo.



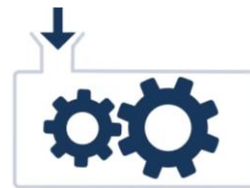


Ejercicios Prácticos

Enunciado 13: Escribe un script en PHP que gestione las notas de un grupo de 3 estudiantes utilizando un arreglo bidimensional. El arreglo debe almacenar el nombre del alumno y sus notas en tres materias (Redes, Programación e Inglés). El script debe:

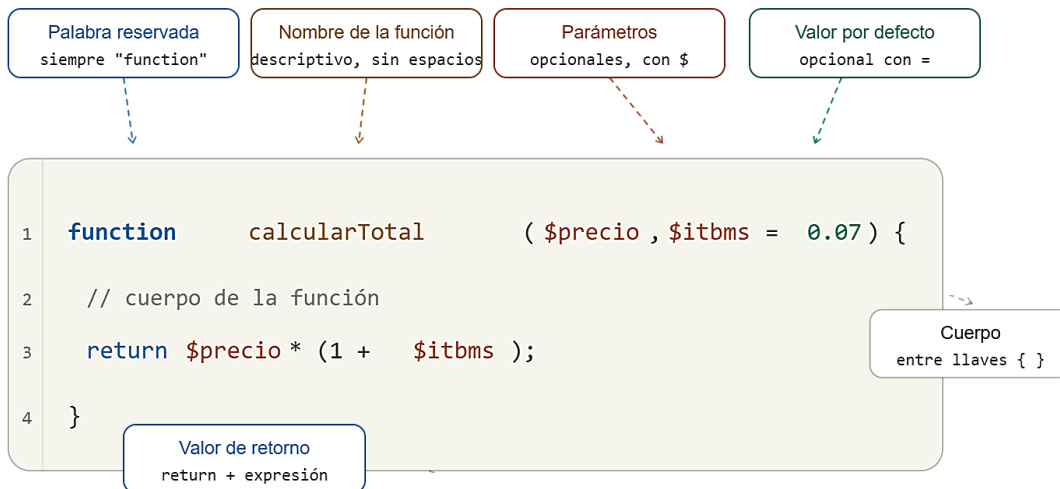
- Recorrer la matriz para mostrar la información de cada alumno.
- Calcular el promedio de notas de cada estudiante.
- Determinar si el estudiante aprobó o reprobó (asumiendo que aprueba con 71 o más).

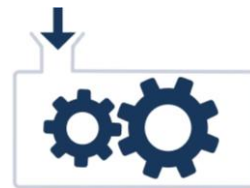




¿Qué es una función?

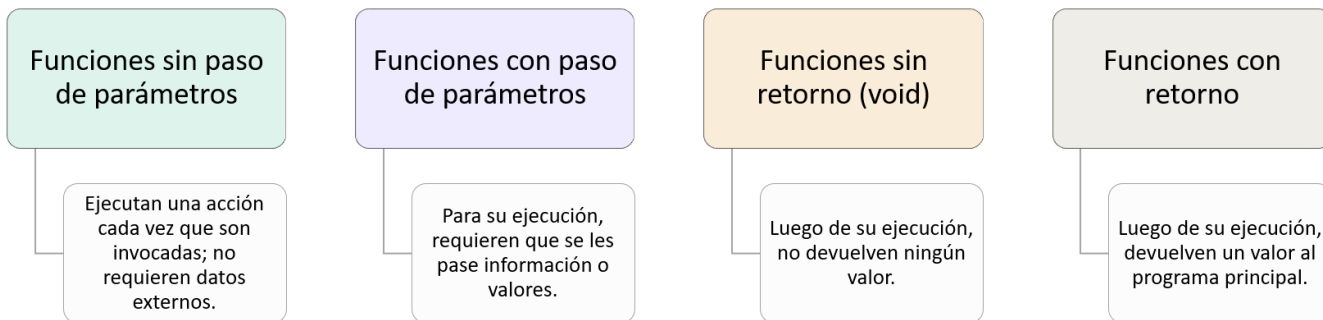
- Una función es un bloque de código reutilizable que realiza una tarea específica. **Principal ventaja:** Nos permiten ahorrar líneas de código.
- En PHP se definen utilizando la palabra clave **function**, seguida del nombre de la función y argumentos entre paréntesis.

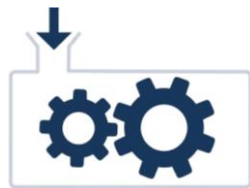




Tipos de funciones

- En general, se distinguen 2 tipos: integradas (built-in) y las definidas por el usuario.
- De acuerdo a su implementación, las funciones se pueden clasificar de la siguiente manera:





Tipos de funciones

TIPO 1

Sin parámetros — sin retorno

No recibe datos externos ni devuelve nada. Ejecuta siempre la misma acción cada vez que se llama.

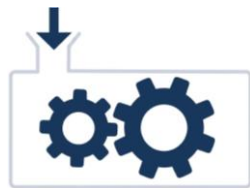
```
function mostrarBienvenida() {  
    echo "Bienvenido al sistema";  
}  
  
mostrarBienvenida(); // Bienvenido al sistema
```

TIPO 2

Con parámetros — sin retorno

Recibe datos externos para personalizar su acción, pero no devuelve nada al programa. El resultado va directo a pantalla o a otro efecto.

```
function saludar($nombre, $hora) {  
    echo "Buenas $hora, $nombre";  
}  
  
saludar("Ana", "tardes");  
// Buenas tardes, Ana
```



Tipos de funciones

TIPO 3

Sin parámetros — con retorno

No necesita datos externos, pero calcula o produce un valor que devuelve al programa para que lo use.

```
function obtenerFecha() {  
    return date("d/m/Y");  
}
```

```
$hoy = obtenerFecha();  
echo "Hoy es: " . $hoy;  
// Hoy es: 13/05/2025
```

TIPO 4

Con parámetros — con retorno

Recibe datos, los procesa y devuelve un resultado. Es la forma más completa y más usada en programación.

```
function calcularIVA($precio, $tasa = 0.16) {  
    return round($precio * $tasa, 2);  
}
```

```
$impuesto = calcularIVA(100);  
echo "IVA: $" . $impuesto;  
// IVA: $16.00
```

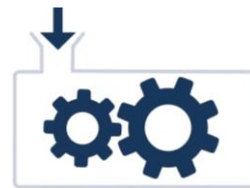


Ejercicios Prácticos

Enunciado 15:

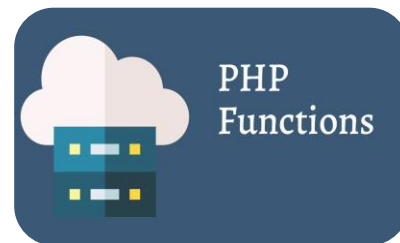
Escribe una función llamada **inic_array** que reciba tres parámetros llamados: `n_elementos`, `min` y `max`, y que devuelva un array de números enteros comprendidos entre los valores `min` y `max`. El número de elementos que contiene el array será el especificado en el parámetro de entrada `n_elementos`.

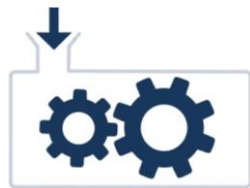




Tipos de funciones (built-in)

- Las funciones **built-in** son aquellas que ya vienen integradas por defecto en PHP.
- PHP dispone de más de 1000 funciones integradas, para realizar tareas comunes, organizadas en diferentes categorías:
 - **Funciones para constantes:** `define()`, `defined()`, `constant()`
 - **Funciones para variables:** `isset()`, `empty()`, `unset()`, `gettype()`, `var_dump()`
 - **Funciones de cadenas:** `strlen()`, `strtolower()`, `explode()`, `str_replace()`, etc.
 - **Funciones matemáticas:** `round()`, `abs()`, `pow()`, `sqrt()`, `rand()`
 - **Funciones de fecha y hora:** `date()`, `time()`, `strtotime()`, `mktime()`

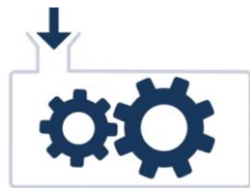




Tipos de funciones (built-in)

- **Funciones para constantes:** `define()`, `defined()`, `constant()`

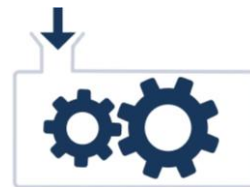
Función	Firma	Descripción
<code>define()</code>	<code>define(name, value)</code>	Define una constante con nombre y valor. <code>define('MAX', 100); // MAX = 100</code>
<code>defined()</code>	<code>defined(name)</code>	Verifica si una constante ya fue definida. <code>defined('MAX') // true</code>
<code>constant()</code>	<code>constant(name)</code>	Obtiene el valor de una constante por su nombre (dinámico). <code>constant('MAX') // 100</code>



Tipos de funciones (built-in)

- **Funciones para variables:** `isset()`, `empty()`, `unset()`, `gettype()`, `var_dump()`

Función	Firma	Descripción
<code>isset()</code>	<code>isset(\$var, ...)</code>	Verifica si una variable está definida y no es null. <code>isset(\$x) // false si no existe</code>
<code>empty()</code>	<code>empty(\$var)</code>	True si la variable es vacía: "", 0, null, false, [], "0". <code>empty("") // true</code>
<code>unset()</code>	<code>unset(\$var, ...)</code>	Destruye una o más variables. <code>unset(\$x); // \$x ya no existe</code>
<code>gettype()</code>	<code>gettype(\$var)</code>	Devuelve el tipo de dato de la variable. <code>gettype(3.14) // "double"</code>
<code>var_dump()</code>	<code>var_dump(\$var, ...)</code>	Muestra tipo y valor detallado (útil para depurar). <code>var_dump([1,2]) // array(2) {...}</code>

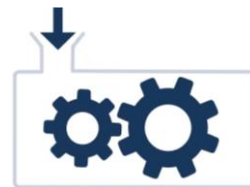


Tipos de funciones (built-in)

- **Funciones matemáticas:** round(), abs(), pow(), sqrt(), rand()

Función	Firma	Descripción
round()	<code>round(\$num, \$prec?)</code>	Redondea al número de decimales indicado. <code>round(4.567, 2) // 4.57</code>
abs()	<code>abs(\$num)</code>	Valor absoluto (quita el signo negativo). <code>abs(-15) // 15</code>
pow()	<code>pow(\$base, \$exp)</code>	Eleva la base a la potencia indicada. <code>pow(2, 8) // 256</code>
sqrt()	<code>sqrt(\$num)</code>	Raíz cuadrada de un número. <code>sqrt(144) // 12.0</code>
rand()	<code>rand(\$min?, \$max?)</code>	Número entero aleatorio entre min y max. <code>rand(1, 100) // ej: 42</code>
ceil()	<code>ceil(\$num)</code>	Redondea siempre hacia arriba. <code>ceil(4.1) // 5</code>
floor()	<code>floor(\$num)</code>	Redondea siempre hacia abajo. <code>floor(4.9) // 4</code>

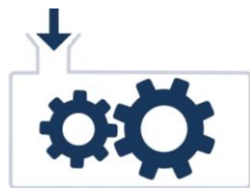
** Las funciones **max()** y **min()** devuelven el valor más alto y el más bajo de un conjunto de valores o de un array.



Tipos de funciones (built-in)

- Funciones de conversión explícita:** intval(), floatval(), strval(), boolval().

Función	Descripción	Tipo de retorno	Ejemplo
intval()	Obtiene el valor entero de una variable. Permite especificar la base (opcional).	int	<code>intval("42")</code> → 42 <code>intval("42.8")</code> → 42 <code>intval(true)</code> → 1
floatval()	Obtiene el valor flotante (decima) de una variable. Es un alias de <code>doubleval()</code> .	float	<code>floatval("3.14")</code> → 3.14 <code>floatval("10")</code> → 10.0 <code>floatval(false)</code> → 0.0
strval()	Convierte una variable a su representación en cadena de texto (<code>string</code>).	string	<code>strval(123)</code> → "123" <code>strval(45.6)</code> → "45.6" <code>strval(123)</code> → "123" <code>strval(45.6)</code> → "45.6" <code>strval(true)</code> → "1" <code>strval(true)</code> → "1"
boolval()	Obtiene el valor booleano de una variable. Disponible desde PHP 5.5.0.	bool	<code>boolval(1)</code> → true <code>boolval(0)</code> → false



Tipos de funciones (built-in)

- **Funciones de fecha y hora:** `date()`, `checkdate()`.

`date()`

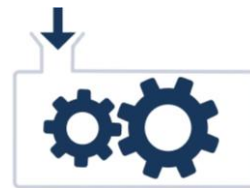
Devuelve la fecha/hora actual formateada como texto. Recibe un patrón de formato.

```
// Fecha completa echo date("d/m/Y"); // 29/05/2025 // Hora actual echo date("H:i:s"); //  
14:35:07 // Fecha y hora juntas echo date("d/m/Y H:i:s"); // 29/05/2025 14:35:07
```

`checkdate()`

Verifica si una fecha es válida. Devuelve true o false.

```
checkdate(2, 29, 2024); // true (2024 es bisiesto) checkdate(2, 29, 2023); // false (2023  
no es bisiesto) checkdate(13, 1, 2025); // false (mes 13 no existe)
```



Tipos de funciones (built-in)

- **Funciones de fecha y hora:** date(), checkdate().

Carácter	Descripción	Ejemplo
d	Día con ceros (01-31)	09
m	Mes con ceros (01-12)	05
Y	Año con 4 dígitos	2025
y	Año con 2 dígitos	25
H	Hora 24h con ceros (00-23)	14
h	Hora 12h con ceros (01-12)	02
i	Minutos con ceros (00-59)	35
s	Segundos con ceros (00-59)	07
A	AM o PM	PM
l	Nombre del día (en inglés)	Thursday
N	Número del día (1=lun, 7=dom)	4



CURSO DE PROGRAMACIÓN VII.

Licenciatura en Desarrollo y Gestión de Software. FISC 2026.